코딩 패러다임의 진화

개발자 중심 구현에서 AI 오케스트레이션까지

Chae Shick Chung

March 10, 2026

강의 목표

이 강의가 끝나면 다음을 설명할 수 있어야 합니다.

- 코딩 방식이 왜 **개발자 직접 구현**에서 **프롬프팅**으로 이동했는지
- 왜 다시 **PRD/TASK 기반 AI 협업** 단계로 진화하는지
- 각 단계에서 필요한 역량(기술/의사소통/검증)
- 실무에 바로 적용할 학습 로드맵

한 장으로 보는 진화

1단계
개발자 → 코딩

⇒

2단계
개발자 → 프롬프팅

⇒

3단계
개발자 → PRD/TASK
→ AI → 코딩

- 핵심 변화: **손 코딩의 비중 감소, 문제정의와 검증의 비중 증가**

1단계의 특징 (전통적 개발)

- 요구사항 해석부터 구현까지 개발자가 대부분 직접 수행
- 언어/프레임워크 문법 숙련도가 생산성의 핵심
- 디버깅, 리팩터링, 테스트 코드도 수작업 중심
- 개인의 구현 속도와 코드 품질 편차가 큼

1단계의 장점과 한계

장점	한계
코드의 동작을 깊이 이해	반복 작업이 많고 시간이 오래 걸림
세밀한 제어 가능	초기 개발 비용이 큼
디버깅 원인 파악이 명확	신규 기술 학습 곡선이 가파름

1단계의 전형적 작업 예시

```
# Requirement: read CSV, handle missing values, print mean
import pandas as pd

df = pd.read_csv("sales.csv")
df["price"] = df["price"].fillna(df["price"].mean())
print(df.groupby("region")["price"].mean())
```

- 모든 로직을 개발자가 직접 설계/작성

2단계의 특징 (Prompt-Driven Coding)

- 개발자가 자연어 지시로 코드 초안을 빠르게 생성
- 구현 속도 급상승
  - **Boilerplate**: 반복적으로 작성하던 기본 코드(라우팅, DTO, 설정, 예외 처리 뼈대) 자동 생성
  - **CRUD**: Create/Read/Update/Delete API와 DB 연동의 표준 패턴을 빠르게 초안화
  - 테스트 초안과 문서 템플릿도 함께 생성 가능
- 사람의 역할이 **작성자**에서 **검토자/수정자**로 이동
- 좋은 프롬프트가 품질을 크게 좌우

프롬프트 품질이 중요한 이유

- 모호한 지시: “로그인 기능 만들어줘”
- 명확한 지시: 입력/출력, 예외, 보안, 테스트 조건 포함

좋은 프롬프트 체크리스트

1. 목표(Goal)
2. 제약(Tech stack, 성능, 보안)
3. 입출력 스펙
4. 검증 기준(테스트/성공조건)

2단계 예시: 프롬프트 기반 생성

```
Prompt:
"Create a FastAPI /login endpoint.
Use JWT, return 401 for bad password,
and add 3 pytest test cases."

AI Output:
- endpoint draft
- token helper
- test drafts
```

2단계에서 자주 생기는 문제

- 그럴듯하지만 틀린 코드(환각)
- 프로젝트 컨텍스트 미반영(기존 아키텍처와 충돌)
- 라이브러리 버전/보안 정책 불일치
- 테스트 없는 복붙으로 기술부채 증가

3단계의 핵심: 구현보다 설계와 운영

- 개발자는 먼저 **PRD(Product Requirements Document)** 작성
- PRD를 실행 가능한 **TASK** 단위로 분해
- AI가 TASK별 코드/테스트/문서를 생성
- 사람은 아키텍처 결정, 품질 게이트, 최종 승인 담당

PRD란 무엇인가?

PRD에는 최소한 다음이 들어가야 합니다.

- 문제 정의(왜 필요한가)
- 사용자 시나리오(누가 어떻게 쓰는가)
- 기능/비기능 요구사항
- 성공 지표(KPI)와 완료 기준(DoD)

TASK 분해 예시

우선순위	Task	완료 조건
P0	인증 API 구현	통합테스트 통과
P0	DB 스키마 마이그레이션	롤백 가능
P1	에러 로깅/모니터링	대시보드 확인
P1	사용자 가이드 문서화	리뷰 승인

3단계 운영 흐름 (실무형)

PRD → Task 분해 → AI 생성 → 테스트/리뷰 → 배포

- 핵심은 **반복 가능한 프로세스**와 **품질 게이트**입니다.

왜 3단계가 중요한가?

- 개인 생산성 최적화가 아니라 **팀 생산성** 최적화 가능
- 지식이 PRD/TASK 문서로 축적되어 온보딩이 쉬움
- 모델이 바뀌어도 프로세스는 재사용 가능
- 코드 품질과 규정 준수(보안/개인정보) 관리 용이

단계별 역량 변화

역량	1단계	2단계	3단계
코딩 문법 숙련	매우 중요	중요	중요
프롬프트 설계	낮음	매우 중요	중요
문제정의/요구사항	중요	중요	매우 중요
테스트/검증 설계	중요	매우 중요	매우 중요
협업/문서화	중간	중간	매우 중요

경제학 관점에서의 의미

- **비교우위 재편**: 단순 구현보다 문제정의/검증 역량의 가치 상승
- **생산성 충격**: 반복 코딩 자동화로 한계비용 감소
- **노동시장 변화**: 주니어 업무 재설계, 시니어의 오케스트레이션 강화
- **조직 설계 변화**: 작은 팀으로도 더 큰 산출 가능

AI 시대 인재상: 도메인 지식 + Vibe Coder

핵심 메시지

도메인 지식이 가장 중요하다면, 이를 AI가 실행 가능한 형태로 번역하는 Vibe Coder 역할이 미래 핵심 인재상이다.

- 도메인 맥락(정책, 산업, 규제, 사용자)을 PRD/TASK로 구조화
- AI 산출물을 도메인 기준으로 검증(타당성, 리스크, 윤리)
- 기술팀-현업팀 사이의 **문제정의 인터페이스** 역할 수행
- 결과적으로 **정확한 문제 설정 능력**이 코딩 생산성을 결정

학습 로드맵 1: 기본기

A. 소프트웨어 엔지니어링

- 자료구조/알고리즘, 네트워크, DB 기본
- API 설계, 에러 처리, 로깅
- Git/GitHub 협업, 코드리뷰 문화

B. 품질 관리

- 단위/통합/E2E 테스트
- 정적분석, 타입체크, CI/CD

학습 로드맵 2: AI 협업 역량

C. 프롬프트 엔지니어링

- 역할 부여, 컨텍스트 제공, 출력 형식 강제
- 실패 케이스 기록 및 프롬프트 템플릿화

D. 컨텍스트 엔지니어링

- PRD/TASK 구조화
- 코드베이스 규칙(아키텍처/스타일) 명시
- RAG/도구호출을 통한 근거 기반 생성

학습 로드맵 3: 실무 적용

E. 실험과 측정

- AI 도입 전후 리드타임 비교
- 결함률/리워크율/테스트 커버리지 측정

F. 거버넌스

- 개인정보/보안 정책
- 라이선스/저작권 이슈
- human-in-the-loop 승인 절차

12주 학습 플랜 (예시)

- 1–4주: 전통 개발 + 테스트 자동화
- 5–8주: 프롬프트 기반 코딩 실습(동일 과제 반복 개선)
- 9–12주: PRD 작성 → TASK 분해 → AI 구현 프로젝트

최종 과제:

- 팀별로 실제 서비스 미니 기능 1개 개발
- 세 단계 방식의 생산성/품질 비교 리포트 제출

PRD 템플릿 (요약)

```
1) Problem: what user pain are we solving?
2) User: who uses it and in what context?
3) Core features: Must / Should / Could
4) Non-functional: performance, security, reliability, cost
5) KPI: success metrics (e.g., latency, conversion)
6) Done criteria: tests, docs, release conditions
```

TASK 카드 템플릿 (요약)

```
Task ID: AUTH-001
Goal: implement email login API
Input: email, password
Output: JWT token
Constraints: <=300ms response, 401 on failure
Validation: 5 pytest cases + security review
Owner: AI draft -> developer review/edit
```

마무리 핵심 메시지

- 코딩의 본질은 "타이핑"이 아니라 "문제 해결"입니다.
- **도메인 지식**을 AI 실행 단위(PRD/TASK)로 바꾸는 역량이 가장 중요합니다.
- AI 시대의 개발자는 **설계자 + 검증자 + 오케스트레이터(Vibe Coder)**로 진화합니다.
- 문서화(PRD/TASK)와 품질게이트가 경쟁력이 됩니다.

Q&A